PROJECT BLUEPRINT & TEAM STUDY GUIDE

Privacy-Aware Clinical Care Platform using Access-Controlled RAG and Continuous Patient Monitoring

A from-zero-to-defence guide for understanding the product, its architecture, data model, APIs, frontend, security boundaries, implementation decisions, known gaps, and likely evaluation questions.

Project context	Details
Team	s^3 — Sheenoj Shakya, Shreya Rajput, Soumya Rathore (from Phase-II report)
Institution	Graphic Era Deemed to be University — Department of Computer Science & Engineering
Primary implementation stack	React + TypeScript, Python + FastAPI, SQLAlchemy, PostgreSQL; Alembic configuration present
Document basis	Available project snapshot `carebridge-current.zip`, Phase-II report, and implementation details confirmed in the conversation
Important scope note	The available ZIP predates some late changes discussed on 21 Oct 2026. The clinical-record and prescription fixes are documented from the later runtime tracebacks and user-confirmed tests; exact current file-level details should be reconciled against the latest local repo.

How to use this guide

Read the guide in order once. Then use the class/file maps as a reference while opening the code. The goal is not memorising lines: it is understanding responsibilities, data flow, trust boundaries, and why each layer exists.

- Part I: product and architecture — what problem the project solves and what is in scope.
- Part II: request flow, security, and data persistence.
- Part III: backend models, schemas, endpoints, frontend modules, and repository files.
- Part IV: tests, limitations, design trade-offs, and viva / defence questions.

Evidence labels used throughout:

Label	Meaning
Confirmed in source snapshot	Directly visible in files inside the available `carebridge-current.zip`.
Confirmed by runtime conversation	A later error log or user-confirmed test established this behaviour.
Planned / not verified	Mentioned in the Phase-II report or proposed architecture, but not proven complete by available source.

Table of contents (guide map)

- 1. Executive summary and problem statement
- 2. Product requirements and actors
- 3. Architecture and technology choices
- 4. Request lifecycle and key workflows
- 5. Security, consent, and access control
- 6. Database design and entity relationships
- 7. Models vs schemas vs API routes
- 8. Backend file-by-file guide
- 9. Model/class reference
- 10. Schema/class reference
- 11. Frontend file-by-file guide
- 12. API inventory and integration patterns
- 13. Clinical records and prescription workflow
- 14. Testing and debugging lessons
- 15. Current status, limitations, and roadmap
- 16. Viva/defence questions with model answers
- 17. Glossary and team study checklist

1. Executive summary

The project is a web-based clinical-care prototype that connects patient symptom intake, provider discovery, appointment workflows, patient records, and doctor-facing actions. The distinctive design goal is privacy-aware sharing: being logged in is not, by itself, sufficient for a clinician to read a patient's protected information. The backend should check the caller's role, the doctor–patient relationship, and the applicable consent/scope before returning or changing protected data.

The stack separates responsibilities. React and TypeScript render the browser experience. Frontend service modules centralise API calls. FastAPI exposes HTTP endpoints, validates requests, authenticates callers, checks authorisation, and orchestrates operations. SQLAlchemy maps Python classes to relational tables. PostgreSQL stores durable data. Alembic configuration and migration files support controlled schema evolution.

The Phase-II report described a prototype, not a clinically certified product. Synthetic/demo data and simulated readings are appropriate for demonstrations. RAG, live wearable connections, emergency Break Glass, full laboratory workflows, and production-grade clinical decision support must not be claimed as complete unless they are separately implemented and tested.

1.1 Problem statement

Clinical information is often spread across disconnected interactions: patients describe symptoms, search for clinicians, book visits, and later need to find records or prescriptions. A connected platform can reduce this fragmentation, but it creates a high-stakes privacy problem: medical data must not become visible to the wrong user merely because an endpoint exists.

1.2 Project objective

- Provide a usable patient and doctor interface for core care workflows.
- Persist profiles, symptoms, appointments, and clinical information in a relational database.
- Enforce authentication and patient–doctor access checks in the backend.
- Support consent-aware sharing and make the authorisation logic explicit.
- Keep future AI/RAG retrieval behind backend permission checks; an LLM must not make the access-control decision.
- Offer a demonstrable academic prototype with clear limitations and testable workflows.

2. Actors, features, and boundaries

Actor	Typical actions	Trust boundary
Patient	Sign in, manage/view own profile, submit symptoms, discover doctors, manage appointments, view clinical records and download own prescriptions.	May access own data. Cannot access another patient's data by changing a URL or ID.
Doctor	Sign in, view eligible linked patients, create clinical records, upload prescriptions, manage appointments/follow-ups where implemented.	Must be authenticated and must pass doctor–patient relationship plus required scope/consent checks.
Backend/API	Validate requests, authenticate identity, authorise operations, query/write database, return safe response schemas.	Trusted enforcement layer; frontend visibility is not security.
Database	Persist users, profiles, relationships, consent, symptoms, appointments and clinical records.	Should not be exposed directly to the browser.
Future AI/RAG component	Retrieve relevant authorised synthetic/clinical documents and generate grounded summaries.	Must receive only documents the backend has already authorised.

2.1 What is in the prototype?

- Authentication/token flow and user roles.
- Patient and doctor profile concepts.
- Symptom intake, symptom catalog/structured submission support in the codebase.
- Doctor directory and appointment workflow.
- Patient timeline and simulated vitals/alert interface modules in the frontend.
- Consent and care-relationship concepts with backend access checks.
- Clinical-record creation and patient-side visibility: confirmed by the later conversation after the missing `clinical_records` table was created.
- Prescription upload, patient-side listing and download: confirmed by the later conversation after aligning Base64 input with PostgreSQL BYTEA storage.

2.2 What must be described cautiously?

- Access-controlled RAG: proposed in the project title/report, but the available source snapshot does not establish a complete production-ready RAG pipeline.
- Continuous monitoring: the UI includes simulated vitals/alerts; do not describe this as live wearable monitoring unless a real device integration exists.
- Audit logging, emergency Break Glass, laboratory workflows, and fine-grained ABAC should be claimed only to the extent demonstrated by the current repository and tests.
- The project is an academic prototype, not a medical device or clinical decision-making system.

3. Architecture and technology choices

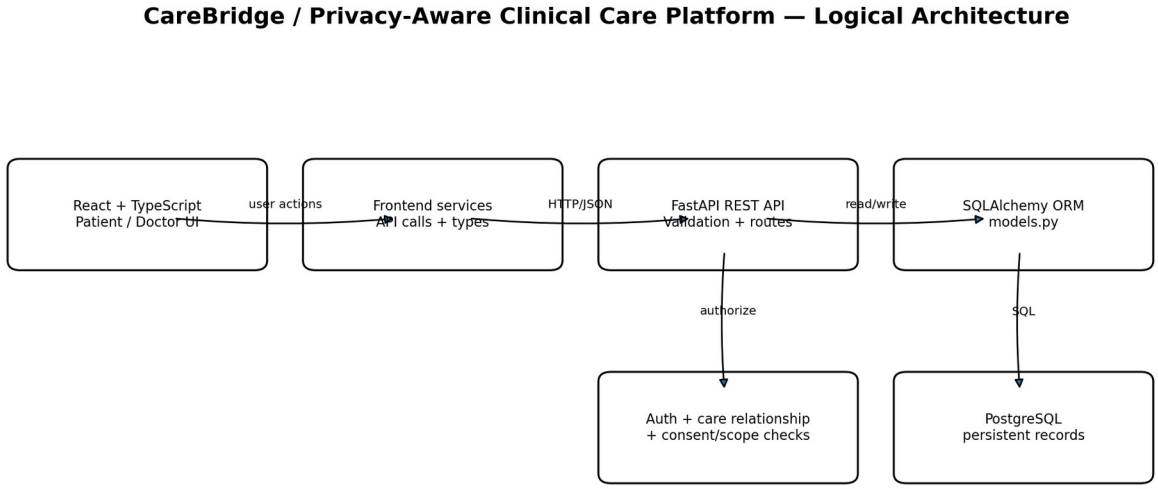


Figure 1. Logical architecture. The frontend sends requests through a service layer to the API. The backend authenticates and authorises the operation before using ORM models to read or write PostgreSQL.

Technology	Role in this project	Why it fits
Python	Backend language.	Readable, broad ecosystem, good support for API/data processing and future AI/RAG

		experimentation.
FastAPI	REST API framework.	Typed request/response models, dependency injection, automatic OpenAPI docs, and efficient route development.
Pydantic	Request/response schema validation (via FastAPI).	Validates incoming data and gives the API a clear contract.
SQLAlchemy 2	ORM and SQL expression layer.	Maps Python classes to tables and lets endpoint code query rows without hand-building SQL for every operation.
PostgreSQL	Relational database.	Transactions, foreign keys, indexes, constraints, and durable relational storage.
Psycopg	PostgreSQL driver.	Connects SQLAlchemy to PostgreSQL.
Alembic	Schema migration tool/configuration.	Records controlled changes to database structure over time; migrations should be used instead of relying only on create_all.
React	Frontend UI library.	Composes dashboard and workflow screens from reusable components.
TypeScript	Frontend language.	Adds static type checking to JavaScript and makes API data contracts clearer.
Vite	Frontend development/build tooling.	Fast local development server and production bundling.
JWT / password hashing	Authentication mechanisms visible in backend dependencies/code.	Allows token-based API authentication without sending passwords on every request; passwords must be stored as hashes, never plaintext.

3.1 Why a modular monolith?

The Phase-II architecture describes a modular monolith: one backend application with logical modules and a single API deployment, rather than multiple separately deployed microservices. This keeps local development, debugging, and a student-team demo manageable. The trade-off is that boundaries must still be maintained through clear modules, schemas, and helper functions.

3.2 Layer separation

Layer	Owns	Should not own
UI/components	Forms, screen state, display, user interactions.	Authoritative access-control decisions or direct database access.
Frontend services	API URL definitions, HTTP calls, token/header handling, conversion to UI data.	Business permission enforcement.

Pydantic schemas	Input validation and response shape.	Database persistence or authorisation policy.
FastAPI routes/helpers	Workflow orchestration, authentication, permission checks, transaction boundary.	UI rendering.
SQLAlchemy models	Table mapping, fields, foreign keys and relationships.	Browser response formatting or UI state.
PostgreSQL	Durable rows, constraints and transactions.	Presentation logic.

4. Request lifecycle: from click to database

1. The user performs an action in a React component (for example, submits a symptom form or uploads a prescription).
2. The component calls a frontend service or the shared API helper.
3. The browser sends an HTTP request to a FastAPI route, usually with JSON and an authentication token; file contents may be encoded as Base64 in the request contract.
4. FastAPI parses and validates the request using a Pydantic schema.
5. Authentication resolves the current user from the token.
6. The endpoint checks role, ownership, doctor–patient relationship, and consent/scope when the operation requires protected patient data.
7. The endpoint uses SQLAlchemy to query or create model instances.
8. The database commits the transaction; the endpoint returns a schema-shaped response.
9. The frontend updates its state or reloads the relevant list so the user sees the persisted result.

Defence point: hiding a button in React is not a security control. A user can call an API directly. The backend must enforce every protected action independently.

5. Security, consent, and access control

5.1 Authentication vs authorisation

Concept	Question answered	Example
Authentication	Who is making the request?	Validate bearer token and resolve `current_user`.
Role-based access control (RBAC)	What broad role does the user have?	A patient cannot call a doctor-only endpoint.
Relationship-based check	Is this doctor linked to this patient?	A doctor cannot access arbitrary patient IDs.
Consent / scope check	Has the patient allowed this category of access?	A relationship may exist, but medical-history access can still require a granted scope.
Ownership check	Does the record belong to the current patient?	Patient prescription download filters by both document ID and patient ID.

5.2 Why multiple checks?

A role check alone is too broad: every doctor account would otherwise be able to read every patient. A relationship check narrows the set of patients, and a consent/scope check narrows what can be viewed for a linked patient. These checks should be centralised in helpers to reduce inconsistent endpoint behaviour.

5.3 Token and password principles

- The login flow creates a signed token with a subject identifier and expiry. The backend reads its configured JWT secret from `backend/.env`.
- Passwords should be verified against an Argon2-compatible hash using the password-hashing library, never compared or stored as plaintext.
- `.env` must not be committed. Share only placeholder variable names in `.env.example`.
- Never include passwords, bearer tokens, database URLs, or real patient data in screenshots, logs, test fixtures, or presentations.

5.4 Security claims and limits

The project includes consent and protected-access concepts, but a commercial healthcare system needs further work: full audit-event coverage, negative access tests for every protected endpoint, secure file storage, key/secret rotation, rate limiting, monitoring, backup/restore testing, threat modelling, and legal/privacy review. A working local prototype is not equivalent to production assurance.

6. Database design and entity relationships

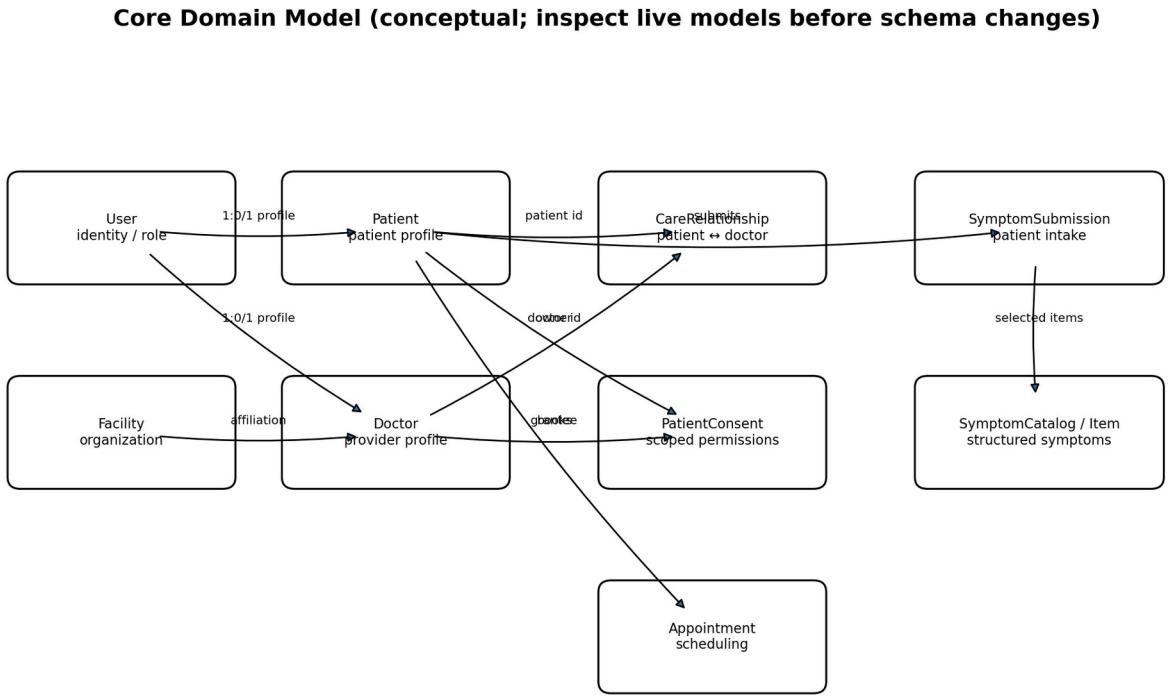


Figure 2. Conceptual view of the core domain. This diagram is explanatory, not a replacement for the actual SQL schema. Confirm foreign keys and nullability in the live `models.py` and migrations before changing the database.

6.1 Core entity purpose

Entity/model	Purpose	Relationship idea
User	Login identity, email/name/role and authentication-related data.	One user may have a patient or doctor profile depending on role.
Patient	Patient-specific profile linked to a user.	Owns symptom submissions, appointments, consent and clinical records.
Doctor	Clinician profile linked to a user; may include specialty/availability fields.	Participates in care relationships and appointments.
Facility	Healthcare organisation/location context.	Can be associated with clinicians and provider discovery.
CareRelationship	Represents a doctor–patient link and its state.	Links a specific patient and doctor; supports relationship-based access.
PatientConsent	Represents consent grants/revocations and access scope.	Connects patient and doctor/recipient to permitted data categories.
SymptomSubmission	Stores a symptom-intake event and narrative/metadata.	Belongs to a patient; may be connected to structured symptom items.
SymptomCatalog	Controlled list of symptoms and catalog metadata.	Referenced by structured symptom submission items.
SymptomSubmissionItem	A selected catalog symptom associated with a submission.	Joins a submission to a catalog entry and can store selected attributes.
Appointment	Scheduled interaction between patient and doctor.	Belongs to one patient and one doctor; has time, reason and status.
ClinicalRecord	Later-added clinical record entity; observed in runtime endpoint and insert SQL.	Belongs to a patient and doctor; record type, title, content, created timestamp.
PrescriptionDocument	Later-added file metadata plus binary payload; confirmed by runtime insert/download flow.	Belongs to patient and doctor; stores filename, content type, file bytes and timestamp.

6.2 What are foreign keys?

A foreign key is a database constraint that says a value must reference a row in another table. For example, `patient_id` in an appointment should refer to a real patient. Foreign keys protect referential integrity: they prevent orphaned records and help make joins reliable.

6.3 Why transactions matter

A SQLAlchemy session groups writes into a transaction. `db.add(record)` stages the object; `db.commit()` attempts to persist it; `db.refresh(record)` reloads database-generated values such as IDs or timestamps. If commit fails, the session should be rolled back before reuse. A failed HTTP request does not mean a record was saved.

6.4 Schema drift and migrations

A model describes the desired mapping in Python; the actual PostgreSQL schema can be older or different. The project encountered exactly this class of issue: the model expected a prescription column named `file_base64`, while PostgreSQL had `file_data` with type BYTEA. The mapping was aligned to the existing column and binary type, and the endpoint decodes the incoming Base64 to bytes. A separate missing-table error for `clinical_records` was resolved by creating the missing model table.

For repeatable team setups, do not depend on manually running `metadata.create_all()` as the only schema-management mechanism. Create and commit an Alembic migration for each intended schema change, test it against a clean database, and document how to upgrade.

7. Models vs schemas vs routes

Term	Definition	Project example	Why separate it?
Model	SQLAlchemy class representing persisted data/table.	`User`, `Patient`, `Appointment`.	Maps application objects to database rows and relationships.
Schema	Pydantic class describing accepted input or returned output.	`LoginRequest`, `AppointmentCreate`, `AppointmentResponse`.	Validates external data and prevents accidentally returning internal columns.
Route/endpoint	HTTP method + path that handles a request.	`POST /patients/me/appointments`.	Connects validation, authentication, business rules, persistence and response.
Service/helper	Reusable function called by routes or UI services.	Frontend `api.ts`; backend access-check helpers.	Avoids duplicating request or policy logic.
Migration	Versioned database schema change.	Files under `backend/migrations/versions/`.	Allows the same schema change to be applied to every environment.

7.1 Example: appointment creation

```
Browser form
  -> AppointmentCreate schema validates doctor_id, scheduled_at, reason
  -> POST /patients/me/appointments
  -> backend identifies patient from current_user (not from arbitrary client patient_id)
  -> validate timezone and future date
  -> create models.Appointment(...)
  -> db.add / commit / refresh
  -> AppointmentResponse returned
  -> frontend displays the confirmed appointment
```

This illustrates why schemas and models are not duplicates. The schema represents the API contract; the model represents the database row. They may overlap in field names, but their purposes differ.

8. Backend file-by-file guide

File / directory	Responsibility and why it exists

`backend/main.py`	FastAPI application entry point. Configures app/middleware, loads environment settings, defines authentication and role helpers, declares routes, validates access, orchestrates database operations and builds responses. It is the API's main control flow.
`backend/models.py`	SQLAlchemy ORM entities. Defines persisted fields, table names, foreign keys, indexes and model relationships. Change this when the Python-to-database mapping changes; pair schema changes with a migration.
`backend/schemas.py`	Pydantic request/response classes. Defines required fields, validation and response contracts. Helps reject malformed input and avoid exposing fields the client should not receive.
`backend/database.py`	Creates/configures the SQLAlchemy engine/session and exposes the database dependency used by routes. Centralises database connection setup.
`backend/requirements.txt`	Python dependency list. Allows the team to install compatible backend libraries in a virtual environment.
`backend/.env.example`	Documents required environment variable names without real secrets. Actual `.env` stays local and ignored.
`backend/alembic.ini`	Configuration for Alembic migration commands.
`backend/migrations/env.py`	Connects Alembic to project metadata/settings so migration operations can target the right database.
`backend/migrations/versions/*.py`	Versioned schema revisions. Each revision should define upgrade and downgrade behaviour for one schema change.
`backend/seed_symptoms.py`	Loads or prepares symptom catalog data for structured intake and demo workflows.
`backend/dev_seed_doctor.py`	Local development helper to create/update a test doctor. It is a dev utility, not a production onboarding flow.
`backend/dev_link_doctor_patient.py`	Local development helper to create a doctor–patient relationship for testing protected workflows. Never use it to bypass real consent in production.
`backend/tests/conftest.py`	Shared test fixtures/setup used by pytest tests.
`backend/tests/test_api_basics.py`	Basic API health/root route checks.
`backend/tests/test_consent_access.py`	Tests around consent/authorisation behaviour; use as a starting point for expanding denied-access coverage.
`README.md`	Root-level setup and project orientation.
`data/`	Project data area; inspect actual files before assuming contents or whether data is synthetic.
`docs/`	Documentation area for project notes and setup

	guidance.
`scripts/`	Helper scripts, if present.

8.1 `main.py`: how to read it

Read this file in functional groups rather than line-by-line on the first pass:

- Configuration and imports: framework dependencies, settings, JWT secret, token expiry, database dependency.
- Application setup: `FastAPI(...)` and CORS middleware. CORS governs which browser origins may call the API; it does not authenticate a user.
- Authentication routes: registration/login and current-user lookup.
- Profile and consent routes: patient/doctor identity, relationships and permissions.
- Symptom routes: free-text and structured symptom intake/catalog.
- Provider/appointment routes: clinician directory, booking, cancellation and doctor decisions.
- Clinical routes added later: clinical record creation/listing, prescription upload/listing/download, and follow-up workflow.

A route commonly follows this pattern: resolve the current user → check role → load the target profile → check relationship/consent/scope → validate payload → query/create model → commit → return safe response.

8.2 `database.py`: key concepts

The SQLAlchemy engine represents the configured database connection pool. A session is a unit of work used by one request/test. The FastAPI dependency typically yields a session and closes it after the request. Never put a long-lived session in a global variable and share it across unrelated requests.

8.3 Dev seed scripts

Development seed scripts are useful because protected workflows need known test accounts and linked test patients. Their hardcoded IDs or local-only assumptions are not production configuration. Keep them untracked or clearly labelled as development-only, never expose their credentials, and do not confuse a seeded relationship with a real patient consent workflow.

9. Backend model/class reference

The available code snapshot defines these model classes:

Class	Core responsibility	What to understand when defending it
`User`	Authentication identity and common account fields.	Role, unique identity fields, password hash, and how it links to a role-specific profile.
`Patient`	Patient profile linked to a user.	Why patient identity is separate from login identity; how patient-owned records are located.
`Facility`	Facility/organisation record.	How a doctor or provider may be associated with a facility.
`Doctor`	Doctor profile and clinician attributes.	How specialty/provider information is used in directory and doctor-only routes.

`CareRelationship`	Doctor–patient relationship state.	Why a relationship is a separate entity instead of a simple boolean on User.
`PatientConsent`	Consent grant/revocation and access scope.	Why consent needs explicit scope and status rather than assuming all linked doctors can see all records.
`SymptomSubmission`	One patient symptom intake event.	Stores the submission as an event so history can be longitudinal.
`SymptomCatalog`	Reference catalogue of structured symptoms.	Controlled choices help standardise intake and support future matching/analytics.
`SymptomSubmissionItem`	Association between a symptom submission and selected catalog item.	Represents the many-to-many-style selection details and attributes.
`Appointment`	Appointment with doctor, time, reason and status.	Status transitions and time validation must be enforced by the backend.

Later runtime evidence adds two entities to the active application path:

Class / entity	Evidence and responsibilities
`ClinicalRecord`	Runtime SQL inserted into `clinical_records` with `patient_id`, `doctor_id`, `record_type`, `title`, and `content`, plus generated ID and `created_at`. The table was missing in the local database and was created from model metadata.
`PrescriptionDocument`	Runtime SQL and endpoints use a prescription document with patient/doctor IDs, filename, content type, binary file payload mapped in Python as `file_base64` to database column `file_data`, and created timestamp.

9.1 Relationships to understand

- User ↔ Patient/Doctor: separates a login account from domain-specific profile data.
- Patient ↔ Appointment ↔ Doctor: one appointment joins one patient and one doctor; a patient and doctor can each have many appointments.
- Patient ↔ CareRelationship ↔ Doctor: represents a set of authorised clinical relationships over time.
- Patient ↔ PatientConsent ↔ Doctor: records the permissions/scopes under which access is allowed.
- Patient ↔ SymptomSubmission ↔ SymptomSubmissionItem ↔ SymptomCatalog: supports repeated intake events with standardised symptom choices.
- Patient ↔ ClinicalRecord ↔ Doctor: a doctor-authored clinical record is attached to a patient and attributed to its author.
- Patient ↔ PrescriptionDocument ↔ Doctor: prescription file belongs to a patient and records which doctor uploaded it.

Use the model declarations and migrations as the authority for exact cardinality, nullability, and cascade behaviour. The conceptual diagram is intended to teach the domain, not replace schema inspection.

10. Schema/class reference

Schema class	Likely role from name and route usage	Defence explanation
`UserCreate`	Registration input.	Validates fields needed to create an account; must not accept arbitrary trusted role values without policy.
`UserResponse`	Public user response.	Returns safe identity fields, not password hash or secrets.
`PatientCreate`	Patient profile input.	Defines accepted patient profile fields.
`PatientResponse`	Patient profile response.	Controls what profile information an endpoint returns.
`LoginRequest`	Login credentials input.	Validates the login request shape.
`PatientConsentCreate`	Consent grant/update input.	Represents patient consent intent and relevant scope/state.
`PatientConsentResponse`	Consent response.	Returns consent status in a client-safe shape.
`SymptomSubmissionCreate`	Free-text symptom submission input.	Validates a patient symptom report.
`SymptomSubmissionResponse`	Symptom submission output.	Returns a safe representation of the saved submission.
`SymptomCatalogResponse`	Catalog item output.	Provides standard symptom choices to the UI.
`StructuredSymptomSubmissionCreate`	Structured intake payload.	Validates selections from the symptom catalog.
`StructuredSymptomSubmissionResponse`	Structured intake output.	Confirms saved structured symptoms and related metadata.
`DoctorDirectoryResponse`	Clinician search/list output.	Keeps directory results distinct from internal Doctor model fields.
`AppointmentCreate`	Appointment creation payload.	Validates requested clinician, timestamp and reason.
`AppointmentResponse`	Patient appointment output.	Represents appointment status and display fields.
`DoctorRegister`	Doctor registration payload.	Captures doctor-specific registration/profile data.
`DoctorProfileResponse`	Doctor profile output.	Defines the clinician profile shape returned to clients.
`DoctorAppointmentResponse`	Doctor-side appointment output.	May include patient-facing appointment display information appropriate for the doctor.
`AppointmentDecision`	Doctor action input for appointment status.	Constrains which decision/status the doctor may request.
`DoctorPatientResponse`	Doctor linked-patient list output.	Returns only patients the doctor is allowed to see.

The available snapshot's `schemas.py` is the source of truth for exact field names and Pydantic options. Later clinical-record/prescription schemas were visible in the runtime conversation as `ClinicalRecordCreate`, `ClinicalRecordResponse`, `PrescriptionCreate`, and `PrescriptionMetadata`; verify their latest field definitions in the current local file.

10.1 Why schemas should not simply return ORM objects everywhere

- They prevent internal columns (such as password hashes or private metadata) from leaking.
- They make API changes explicit and reviewable.
- They validate user-supplied data before business logic runs.
- They decouple the public API contract from internal database refactoring.
- For file downloads, the response contract must encode binary bytes as a transport-safe representation such as Base64 or stream the file with an appropriate content type.

11. Frontend file-by-file guide

File	Purpose / reason it exists
`frontend/src/main.tsx`	React entry point. Mounts the app into the HTML root and loads global styles.
`frontend/src/App.tsx`	Top-level application composition, navigation, authentication/role-aware page layout, and integration of feature screens. Read this to understand how the whole UI is assembled.
`frontend/src/App.css`	Application-level component/layout styling.
`frontend/src/index.css`	Global/reset styles and base typography/layout rules.
`frontend/src/services/api.ts`	Shared HTTP helper; central place for base URL, JSON handling, auth headers and API error handling.
`frontend/src/services/authService.ts`	Authentication-specific requests/state helpers such as login/current-user.
`frontend/src/services/endpoints.ts`	Named endpoint constants so route strings are not scattered through components.
`frontend/src/services/patientService.ts`	Patient-related API operations grouped in one service.
`frontend/src/services/providerService.ts`	Provider/doctor directory requests.
`frontend/src/services/mockdata.ts`	Synthetic/fallback sample data for development/demo contexts; confirm whether each consumer still uses it.
`frontend/src/types/healthcare.ts`	Shared TypeScript interfaces/types for healthcare entities and API data shapes.
`frontend/src/features/symptoms/SymptomIntake.tsx`	Symptom intake UI, including form state, structured/free-text inputs and submission flow.
`frontend/src/features/doctors/FindDoctor.tsx`	Doctor discovery/search UI.
`frontend/src/features/appointments/Appointments.tsx`	Appointment list and booking/status interaction UI.
`frontend/src/features/providers/ClinicianDashboard.tsx`	Clinician/doctor-facing dashboard module.
`frontend/src/features/timeline/`	Patient timeline/history view.

PatientTimeline.tsx`	
`frontend/src/features/monitoring/ VitalsAlerts.tsx`	Vitals/alert UI; report indicates simulated readings rather than confirmed live wearable integration.
`frontend/package.json`	Frontend scripts and dependencies.
`frontend/package-lock.json`	Locks the resolved npm dependency tree for reproducible installs.
`frontend/vite.config.ts`	Vite dev server/build configuration.
`frontend/tsconfig*.json`	TypeScript compiler configuration for application and tooling.
`frontend/index.html`	HTML shell into which the React app mounts.
`frontend/public/`	Static assets served without module imports.

11.1 How React component state fits in

A component holds transient UI state such as form values, loading flags, validation errors, selected patient, or fetched results. Durable clinical data belongs in the backend/database. After a successful write, the UI should refresh the relevant list or update it from the server response. The database is the source of truth; a local React array alone is not persistence.

11.2 TypeScript types

An interface such as an appointment type tells the compiler what fields the UI expects. It catches many mismatches during development, but it does not validate untrusted HTTP data at runtime. Backend Pydantic validation remains necessary.

12. API inventory and integration patterns

The following routes were found in the available source snapshot. The exact route list may be larger in the current local version because clinical-record, prescription and follow-up endpoints were added later.

HTTP method	Route path
GET	/
GET	/health
POST	/auth/login
POST	/auth/register/doctor", response_model=UserResponse, status_code=status.HTTP_201_CREATED
GET	/auth/me", response_model=UserResponse
GET	/patients/me", response_model=PatientResponse
GET	/doctors/patients/{patient_id}/profile", response_model=PatientResponse
GET	/doctors/directory", response_model=list[DoctorDirectoryResponse]
POST	/patients/me/appointments", response_model=AppointmentResponse, status_code=status.HTTP_201_CREATED
GET	/patients/me/appointments",

	response_model=list[AppointmentResponse]
POST	/patients/me/appointments/{appointment_id}/cancel", response_model=AppointmentResponse
GET	/doctors/me/profile", response_model=DoctorProfileResponse
GET	/doctors/me/appointments", response_model=list[DoctorAppointmentResponse]
PATCH	/doctors/me/appointments/{appointment_id}", response_model=DoctorAppointmentResponse
GET	/doctors/me/patients", response_model=list[DoctorPatientResponse]

12.1 How to interpret HTTP methods

Method	Typical intent	Example
GET	Read data without changing server state.	Read profile or appointment list.
POST	Create a resource or perform an action.	Submit symptoms or create appointment.
PATCH	Partially update an existing resource.	Doctor changes appointment status.
PUT	Replace a resource (if used).	Full update semantics.
DELETE	Remove a resource (if used).	Deletion where the API supports it.

12.2 API response codes

- 200 OK: successful read/update.
- 201 Created: resource created successfully.
- 401 Unauthorized: missing/invalid authentication.
- 403 Forbidden: authenticated user lacks permission.
- 404 Not Found: requested resource does not exist or is intentionally hidden from caller.
- 409 Conflict: state conflict such as an already occupied appointment slot.
- 422 Unprocessable Entity: payload fails validation or violates request-level rules.
- 500 Internal Server Error: unexpected server/database problem; inspect the backend traceback, not only the browser message.

12.3 Why central endpoint constants matter

A named endpoint map in `endpoints.ts` avoids duplicating route strings across screens. If a path changes, one central constant can be updated. It also makes integration review easier: the frontend contract can be compared against FastAPI route declarations.

13. Clinical records and prescriptions: end-to-end explanation

13.1 Clinical-record creation

13. Clinical records and prescriptions: end-to-end explanation

13.1 Clinical-record creation

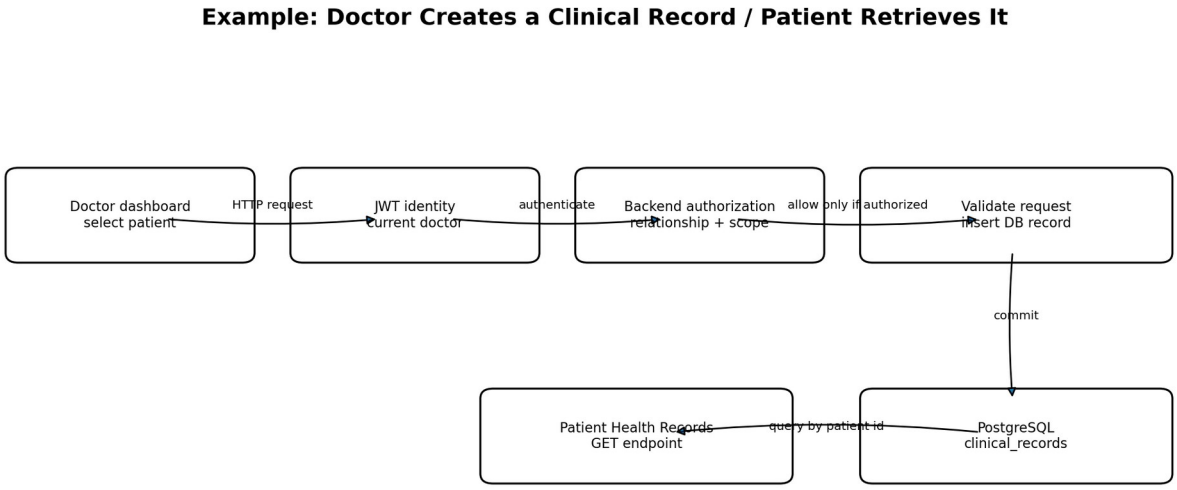


Figure 3. A conceptual clinical-record flow. A doctor request is authorised before a record is saved. The patient endpoint later queries records belonging to the authenticated patient.

- The doctor-facing request uses the patient identifier in the URL and a payload with record type/title/content.
- The backend resolves the current doctor and checks the doctor–patient relationship and the medical-history scope.
- The endpoint creates a `ClinicalRecord` row with patient ID, doctor ID, record type, title and content.
- The database commit is the persistence boundary. If the table is missing, the insert fails and the record is not saved.
- The patient endpoint identifies the patient from the authenticated user and queries records by that patient's ID; the frontend displays returned records.

13.2 The missing-table incident and what it taught us

A runtime traceback reported `relation "clinical_records" does not exist`. The ORM attempted an INSERT, but the local PostgreSQL schema did not yet contain the table. Running `models.Base.metadata.create_all(bind=engine)` created missing model tables in that local database, after which the user confirmed the workflow worked. This is a schema deployment issue, not necessarily a UI rendering issue.

Long-term fix: add a migration that creates `clinical_records`, apply it to all environments, and include a test that starts from an empty database. `create_all()` does not update existing tables and should not replace migration discipline.

13.3 Prescription upload/download

- The browser sends file metadata plus Base64-encoded content as the upload payload.
- The backend validates Base64 and basic file signatures/size before storing the content.
- Base64 is a text encoding of bytes; PostgreSQL BYTEA stores raw bytes. The upload path therefore decodes Base64 before persistence.
- The ORM attribute is mapped to the existing database column: Python `file_base64` ↔️ SQL column `file_data`, with SQLAlchemy `LargeBinary` to match PostgreSQL BYTEA.
- The patient list endpoint returns metadata. The download endpoint encodes stored bytes back to Base64 for the frontend contract.

The runtime sequence exposed three distinct failure classes: missing `file_base64` column; existing `file_data` NOT NULL column; then a type mismatch because `file_data` was BYTEA but the app sent a string. The final fix aligned the Python attribute/column name, used `LargeBinary`, decoded input bytes before insert, and encoded bytes when returning a Base64 JSON field.

13.4 Binary file formats in plain language

Term	Meaning
Raw bytes	The actual binary contents of the file in memory.
Base64	A text representation of binary data that can be embedded in JSON; increases payload size by roughly one third.
BYTEA	PostgreSQL data type for binary byte sequences.
LargeBinary	SQLAlchemy type that maps Python bytes to a database binary type.
Content type	A MIME type such as `image/png`, `image/jpeg`, or `application/pdf` that tells the client how to handle the file.

Production consideration: storing files in a database can be valid for a small prototype, but a sellable healthcare product should evaluate private object storage, size limits, malware scanning, signed/authorised downloads, encryption, audit events, backup and retention policy. Never make clinical documents public.

14. Testing, debugging, and reliability

14.1 Tests present in the available snapshot

Test file	What it is intended to cover
`backend/tests/test_api_basics.py`	Basic route/app behaviour.
`backend/tests/test_consent_access.py`	Consent and access-control scenarios.
`backend/tests/conftest.py`	Reusable test setup and fixtures.

An earlier project run reported 9 passed and 1 warning, but this is historical evidence from the earlier snapshot. Do not quote it as proof that the latest clinical-record and prescription changes have been fully tested. Run the suite against the current working tree before final submission.

14.2 Debugging playbook

Symptom	What it usually means	First investigation
Browser says Failed to fetch	Backend down, wrong port/base URL, CORS, network or request blocked.	Check server terminal, browser Network tab, API base URL and CORS origin.
401	Token missing/expired/invalid.	Check login flow, auth header and JWT settings; never paste the token publicly.
403	Caller authenticated but not authorised.	Check role, relationship and consent/scope policy.
404	Resource or route not found; may also deliberately hide protected resources.	Check route path, resource ID, ownership filter and DB data.
422	Payload schema/rule failure.	Read response detail; compare JSON fields, types, timezone and file validation.
500 with UndefinedTable	The table is missing in the database selected by the app.	Check active DATABASE_URL/schema and apply migration; don't blindly change UI.
500 with UndefinedColumn	Model and DB column names differ.	Compare ORM mapping with actual DB columns and migration history.
DatatypeMismatch BYTEA vs VARCHAR	App sends a string to a binary column.	Decode Base64 before save and use LargeBinary mapping.
Save succeeds but UI list is empty	Retrieval endpoint, ownership filter, consent filter, API call or UI refresh may be wrong.	Check GET endpoint response in Network tab, query filter and state refresh.

14.3 Reliable fix workflow

10. Read the first meaningful exception at the top of the traceback; the final SQLAlchemy error often wraps the underlying PostgreSQL error.
11. Identify which layer failed: UI, HTTP request, validation, auth, ORM mapping, database schema, or response rendering.
12. Make the smallest change that aligns the contracts; avoid adding random columns or weakening access checks.
13. Restart/reload the backend and reproduce the exact action.
14. Verify the write in the UI and by refreshing/relogging; success should survive a reload.
15. Test both allowed and denied cases, not only the happy path.
16. Record the fix and add a regression test or migration.

15. Current status, limitations, and roadmap

Area	Status based on available evidence	What is still needed
Patient/doctor UI	Present in React feature modules	Validate all screens in the latest

	and screenshots.	build and improve responsive/accessibility polish.
Authentication	Token-based auth and user roles visible in backend.	Review token expiry/refresh strategy, rate limiting, account recovery and threat model.
Symptoms/catalog	Free-text and structured symptom modules/routes present.	Re-test specialty recommendation logic; prior conversation said recommendations were still an issue.
Provider directory/appointments	Routes and frontend modules present.	Retest timezone handling, conflicts, cancellation/status transitions and empty states.
Clinical records	Later runtime flow confirmed working after missing table was created.	Add Alembic migration and regression tests; test consent-denied paths.
Prescription upload/list/download	Later runtime flow confirmed visible and downloadable by patient.	Test MIME/type/size validation, wrong-patient access, corrupt files and file persistence; reconcile schema via migration.
Consent/care relationship	Model/routes and access test file present.	Audit every protected endpoint and test revoked/absent consent.
Vitals/alerts	Frontend monitoring module present; Phase-II report describes simulated readings.	Don't imply live hardware integration unless implemented; add alert acknowledgement/resolution if required.
Access-controlled RAG	Project goal/plan, not established as complete by available code snapshot.	Implement retrieval only after authorisation, chunking/embeddings/vector store, source citations, and allow/deny tests.
Audit trail	Described in the report/target architecture; full implementation not verified in snapshot.	Confirm event model and coverage; audit sensitive reads, changes, exports and denied access.
Production readiness	Not established.	Security review, privacy/legal review, observability, backups, migrations, deployment secrets, load tests, disaster recovery.

15.1 Roadmap prioritisation

- Priority 1 — stabilise current flows: auth, symptom submission, doctor search, appointments, clinical records, prescription upload/download.
- Priority 2 — database repeatability: Alembic migrations for every model/table and a clean-database setup test.
- Priority 3 — privacy assurance: negative access tests for each protected route, consent revocation behaviour, audit coverage.
- Priority 4 — UX and reliability: clear errors, loading/empty states, refresh persistence, responsive layouts.

- Priority 5 — future features: permission-aware RAG, richer monitoring, lab workflows, alert lifecycle and integrations.

15.2 Production caution

Because this project handles health-related information, a working demo must not be marketed as clinically safe or compliant without a formal assessment. Use synthetic data in demonstrations. Before real deployment, get security, privacy, legal, and clinical-domain review and define the jurisdiction-specific obligations.

16. Viva / defence questions and suggested answers

Q. Why did you choose FastAPI?

A. It provides typed request handling, dependency injection, automatic OpenAPI documentation, and a straightforward way to organise REST endpoints in Python.

Q. Why React and TypeScript?

A. React supports component-based interactive screens. TypeScript catches many UI/API contract errors during development and makes data shapes explicit.

Q. What is SQLAlchemy?

A. It is the ORM layer: Python classes represent database tables and objects represent rows. It also supports expressive SQL queries and transaction management.

Q. What is the difference between a model and a schema?

A. A model maps persisted data to the database. A schema validates incoming/outgoing API data. They serve different trust boundaries and should not be conflated.

Q. Why PostgreSQL instead of storing everything in JSON files?

A. The platform needs durable relational records, foreign keys, transactions, constraints and queries across patients, doctors, appointments and records.

Q. Why is login not enough to access a patient's record?

A. Authentication only identifies the caller. Authorisation must also check role, patient–doctor relationship and applicable consent/scope.

Q. Why does the backend need access checks if the UI hides restricted patients?

A. Client-side UI can be bypassed. Every protected endpoint must enforce access on the server.

Q. What does consent-aware access mean?

A. Access is evaluated against a patient's explicit permissions and scope, not merely the fact that a doctor exists or is logged in.

Q. What caused the clinical-record 500 error?

A. The ORM attempted to insert into `clinical_records`, but that table did not exist in the active PostgreSQL database. The missing table was created; a migration should make the fix repeatable.

Q. Why did prescription upload have a BYTEA error?

A. The database column was binary BYTEA but the app supplied Base64 text. The upload now decodes Base64 to bytes, and SQLAlchemy maps the Python attribute to the existing `file_data` column using LargeBinary.

Q. Why encode the downloaded bytes as Base64?

A. The API returns JSON; Base64 makes binary content representable as text. Alternatively, a dedicated file response could stream bytes with the correct content type.

Q. What is an Alembic migration?

A. A versioned script that changes the database schema in a controlled, repeatable way. It keeps local, test and deployment databases aligned.

Q. What does `db.commit()` do?

A. It asks the database to commit the session's transaction. If it fails, the write did not complete and the session should be rolled back before reuse.

Q. What is a foreign key?

A. A constraint linking a column to a row in another table, enforcing referential integrity.

Q. Is the RAG feature complete?

A. Do not claim it is complete unless the current code and a working demonstration prove it. The Phase-II report listed permission-aware RAG as a planned/stretch feature.

Q. Is monitoring connected to real wearables?

A. The report describes simulated readings. Say simulated unless the current code demonstrates a real device integration.

Q. How would you improve this project next?

A. Prioritise migrations, protected-route negative tests, consistent audit logging, reliable file handling, observability and deployment security before expanding AI features.

17. Glossary

Term	Plain-language meaning
API	A defined way for one program to request data/actions from another.
REST	A common style for HTTP APIs built around resources and methods.
Endpoint	One method + URL path handled by the backend.
ORM	Object-relational mapper; maps classes/objects to relational tables/rows.
Pydantic schema	Typed data contract that validates input or shapes output.
JWT	Signed token carrying claims such as user identity and expiry.
RBAC	Role-based access control.
ABAC	Attribute-based access control; policies consider attributes such as role, relationship, consent scope, context.
Consent scope	The category of data/actions a patient permits a clinician to access.
Foreign key	Database reference from one row/table to another.
Migration	Versioned change to database schema.

Transaction	A group of database operations committed or rolled back together.
CORS	Browser policy controlling which origins may make cross-origin requests; not an auth system.
Base64	Text encoding for binary data.
BYTEA	PostgreSQL binary data type.
RAG	Retrieval-augmented generation: retrieve source documents and provide them as context to a language model.
Embedding	Numeric vector representation used for semantic similarity search.
pgvector	PostgreSQL extension for storing/searching vectors; planned unless verified in current code.
Synthetic data	Artificial data used for testing/demos rather than actual patient information.
Break Glass	Emergency access mechanism that may temporarily override normal access with strict justification and auditing; planned unless verified.

18. Team study plan: from zero to defence-ready

17. Round 1 — Product story: each member explains the problem, target users, core workflows, and why privacy is a first-class requirement.
18. Round 2 — Trace a request: open one frontend component, follow its service call, locate the FastAPI route, schema, model, and database write.
19. Round 3 — Explain the data model: draw User, Patient, Doctor, CareRelationship, PatientConsent, Appointment, SymptomSubmission, ClinicalRecord, and PrescriptionDocument from memory.
20. Round 4 — Security drill: explain what happens for an authenticated doctor with no active relationship, a revoked consent, or a patient trying to download another patient's file.
21. Round 5 — Debugging drill: explain the missing-table and BYTEA/Base64 errors and why each fix belonged to a different layer.
22. Round 6 — Demo rehearsal: demonstrate only working flows, use synthetic accounts/data, refresh after writes, and have a fallback if the local server fails.
23. Round 7 — Evidence check: run the current backend tests and frontend build, verify database migrations, and update this document where the repository has changed.

Appendix A. Source inventory captured from available project ZIP

The file inventory below was extracted from the available `carebridge-current.zip` snapshot. Files under caches and bytecode are excluded. Because the user confirmed later code changes after that archive was created, this inventory is a snapshot, not a guarantee of the latest working tree.

Backend files

- backend/.env.example

- backend/alembic.ini
- backend/database.py
- backend/dev_link_doctor_patient.py
- backend/dev_seed_doctor.py
- backend/main.py
- backend/migrations/README
- backend/migrations/env.py
- backend/migrations/script.py.mako
- backend/migrations/versions/01e2500f022d_add_symptom_submissions.py
- backend/migrations/versions/89c8db9a4c8a_add_user_roles.py
- backend/migrations/versions/98abd5fe8474_add_patient_doctor_facility_.py
- backend/migrations/versions/9cca93118f4c_add_patient_consent.py
- backend/migrations/versions/b0d1c2e3f4a5_allow_appointment_approval.py
- backend/migrations/versions/d54e03995f3b_add_structured_symptom_catalog.py
- backend/migrations/versions/e61c90b2a3f1_add_appointments.py
- backend/migrations/versions/f252378e7711_create_users_table.py
- backend/models.py
- backend/requirements.txt
- backend/schemas.py
- backend/seed_symptoms.py
- backend/tests/conftest.py
- backend/tests/test_api_basics.py
- backend/tests/test_consent_access.py

Frontend source files

- frontend/src/App.css
- frontend/src/App.tsx
- frontend/src/assets/hero.png
- frontend/src/assets/react.svg
- frontend/src/assets/vite.svg
- frontend/src/features/appointments/Appointments.tsx
- frontend/src/features/doctors/FindDoctor.tsx
- frontend/src/features/monitoring/VitalsAlerts.tsx
- frontend/src/features/providers/ClinicianDashboard.tsx
- frontend/src/features/symptoms/SymptomIntake.tsx
- frontend/src/features/timeline/PatientTimeline.tsx
- frontend/src/index.css
- frontend/src/main.tsx
- frontend/src/services/api.ts
- frontend/src/services/authService.ts
- frontend/src/services/endpoints.ts
- frontend/src/services/mockdata.ts
- frontend/src/services/patientService.ts
- frontend/src/services/providerService.ts
- frontend/src/types/healthcare.ts

Appendix B. Environment and run commands

Typical local commands based on the project workflow; run them from the indicated directories and use the active virtual environment.

```
# Backend (PowerShell)
cd C:\projects\healthcare-platform\backend
python -m uvicorn main:app --reload

# Run backend tests (from backend folder)
python -m pytest -q

# Frontend (in a separate terminal)
cd C:\projects\healthcare-platform\frontend
npm install
npm run dev

# Production bundle check
npm run build
```

Never paste actual `.env` values, passwords, JWT secrets, or access tokens into documentation or screenshots.

Appendix C. Evidence and source limits

The Phase-II report provides the project title, team, planned architecture, scope, risks, and roadmap. The available ZIP provides the file/class/route inventory used in this guide. The latest clinical-record and prescription behaviour is grounded in runtime tracebacks and the user's confirmation in conversation. The source ZIP appears older than those late changes, so compare every file-level statement against the current local repository before treating this as a definitive source-of-truth manual.

This blueprint is intended as a study and defence aid, not a substitute for reading the code. When exact behaviour matters, the current implementation and migrations take precedence over this explanatory document.